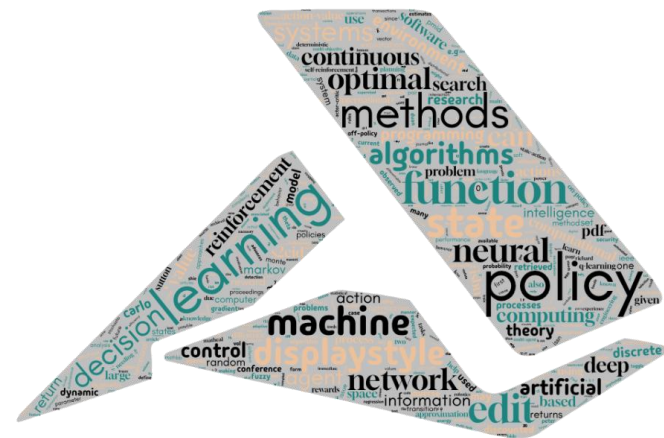


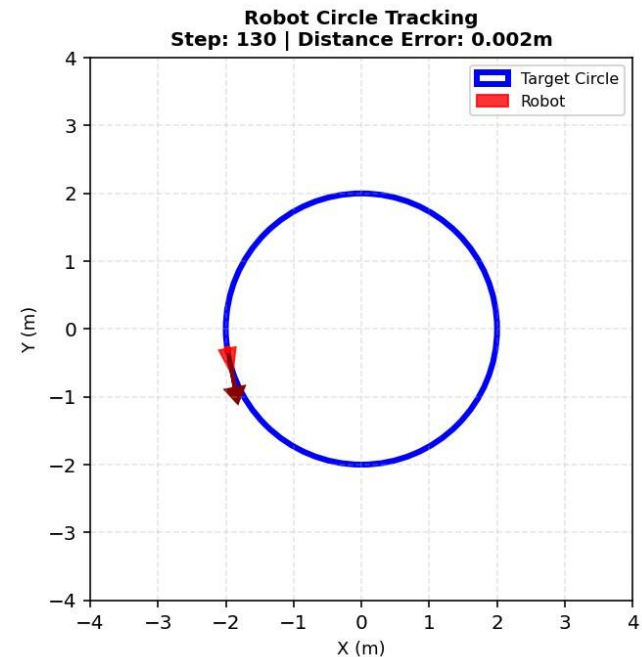


Reinforcement Learning Algorithms and Applications

Chapter 13. SAC for Mobile Robot Line Tracking

Problem Setup

- ▶ **Circle Tracking with Differential Drive Robot**
- ▶ **Environment:** 2-wheeled mobile robot tracking circular trajectory
- ▶ **State Space:** $[dx, dy, d\theta, \text{dist_to_circle}]$ (4D continuous)
- ▶ **Action Space:** $[\text{linear_vel}, \text{angular_vel}]$ (2D continuous)
 - ▶ Linear velocity: $[0, 1.0]$ m/s (forward only)
 - ▶ Angular velocity: $[-1.0, 1.0]$ rad/s
- ▶ **Training Parameters:**
 - ▶ Total timesteps: 300,000
 - ▶ Max steps per episode: 200
 - ▶ Replay buffer: 200,000
 - ▶ Learning rate: 5×10^{-4}



Soft Actor-Critic (SAC)

Why use log standard deviation instead of standard deviation in SAC?

- ▶ The standard deviation of a Gaussian distribution must always be positive: $\sigma > 0$
- ▶ But neural networks can output any real number: $(-\infty, +\infty)$
- ▶ If the network directly outputs std, it may produce invalid values like: $std = -3$ which is not mathematically valid.

Solution: Output log standard deviation

- ▶ Instead of predicting std, SAC predicts: $\log(\sigma)$
- ▶ Convert it back using: $\sigma = e^{\log(\sigma)}$
- ▶ Since: $e^x > 0$ the standard deviation is always positive.

The log space is smoother and more compressed

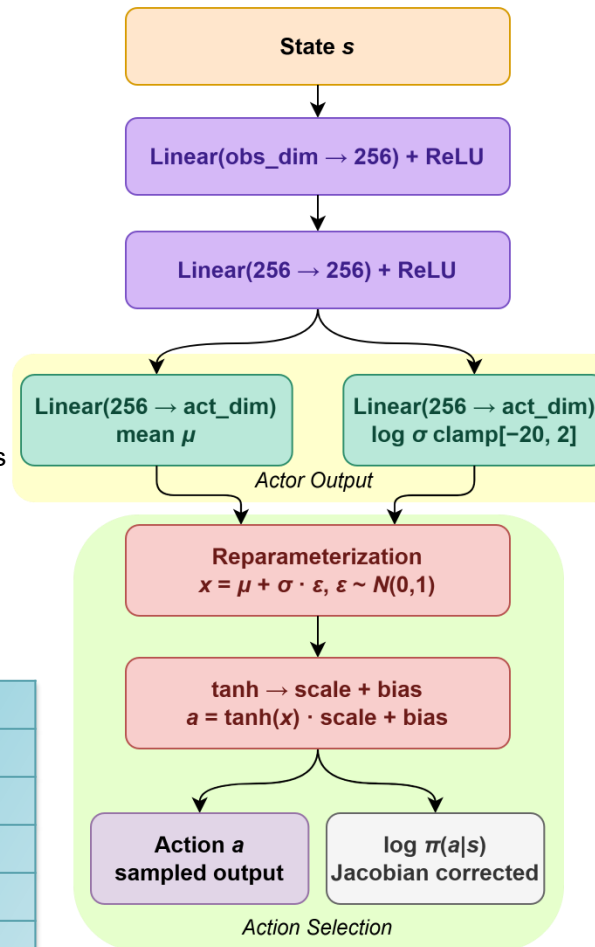
- ▶ Large changes in std become smaller changes in log standard deviation

Benefits for neural network training

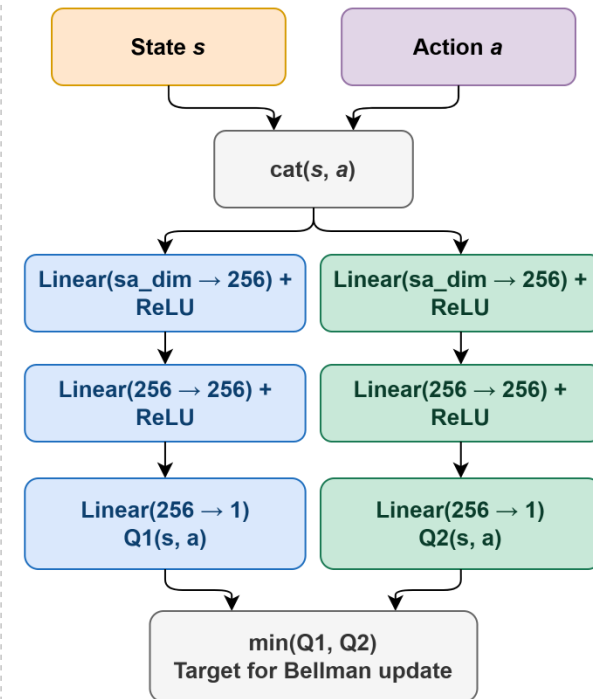
- ▶ More stable gradients
- ▶ Easier optimization
- ▶ Prevents sudden huge changes in exploration
- ▶ Better numerical stability

Standard deviation	Log standard deviation
0.001	-6.9
0.01	-4.6
0.1	-2.3
1	0
10	2.3

Actor (Policy Network)



Critic (Double Q-Network)



Soft Actor-Critic (SAC)

- ▶ **Discrete Entropy**

- ▶ $H(\pi) = -\sum_a \pi(a | s) \log \pi(a | s)$

- ▶ **Continuous Entropy**

- ▶ $H(\pi) = \mathbb{E}[-\log \pi(a | s)]$

- ▶ Closed-form for Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$

- ▶ $H(\pi) = \frac{1}{2} \log(2\pi e \sigma^2)$

- ▶ Sample-based entropy estimator

- ▶ We use this, because tanh is included (Closed-form for Gaussian distribution cannot be applied).

- ▶ $H(\pi) = -\log \pi(a | s)$

- ▶ **Target Q-value:**

- ▶ $y = r + \gamma \left[\min \left(Q_{\text{target}_1}(s', a'), Q_{\text{target}_2}(s', a') \right) - \alpha \log \pi_\theta(a' | s') \right]$

- ▶ Because the target y already includes the entropy term, the Q-network ultimately learns a soft Q-value that incorporates the entropy bonus.

- ▶ **Critic loss:**

- ▶ $L_{\text{critic}} = \frac{1}{N} \sum_{i=1}^N [(Q_1(s_i, a_i) - y_i)^2 + (Q_2(s_i, a_i) - y_i)^2]$

- ▶ **Actor Loss**

- ▶ $L_{\text{actor}} = -\frac{1}{N} \sum_{i=1}^N [\min(Q_1(s_i, a_i), Q_2(s_i, a_i)) - \alpha \log \pi_\theta(a_i | s_i)]$

- ▶ If training become unstable, we can only use $Q_1(s_i, a_i)$ for the actor loss.

- ▶ **Entropy Coefficient (α)**

- ▶ High $\alpha \rightarrow$ Explore more

- ▶ Low $\alpha \rightarrow$ Exploit more

- ▶ In this case, we choose a fixed value ($\alpha = 0.2$).

Soft Actor-Critic (SAC)

► Policy Sampling (Reparameterization)

- $[\mu_i, \log(\sigma_i)] \leftarrow \text{Actor}(s_i)$

- $\sigma_i = e^{\log(\sigma_i)}$

► Reparameterization Trick

- Ideally, we want:

- $x_i \sim \mathcal{N}(\mu_i, \sigma_i) = \mathcal{N}(\mu_i, e^{\log(\sigma_i)})$

- Makes sampling differentiable (backpropagation works)

- $\epsilon_i \sim \mathcal{N}(0, 1)$

- $x_i = \mu_i + \sigma_i \epsilon_i$

► Action Scaling

- $a_i = w \tanh(x_i) + b$

- Tanh ensures the value within $[-1, 1]$

- w (action scale) and b (action bias) transform the action value to a suitable range

► Jacobian Correction

- Because the final action is transformed from x_i to a_i , the probability density changes.

► Probability Density Transformation

- $\pi(a_i | s_i) = \pi(x_i | s_i) \left| \frac{dx_i}{da_i} \right|$

- $\log \pi(a_i | s_i) = \log \pi(x_i | s_i) + \log \left| \frac{dx_i}{da_i} \right| = \log \pi(x_i | s_i) - \log \left| \frac{da_i}{dx_i} \right|$

► Compute the Derivative

- $a_i = w \tanh(x_i) + b$

- $\left| \frac{da_i}{dx_i} \right| = |w(1 - \tanh^2(x_i))| = w(1 - \tanh^2(x_i))$ Because $-1 < \tanh(x_i) < 1$, $1 - \tanh^2(x_i) > 0$.

► Final Log-Probability (Use for calculating Actor loss)

- For multi-dimensional actions, log of joint probability = sum of log-probabilities

- $\log \pi(a_i | s_i) = \sum_j [\log \mathcal{N}(x_{i,j}; \mu_{i,j}, \sigma_{i,j})] - \sum_j [w_j (1 - \tanh^2(x_{i,j}))]$

Soft Actor-Critic (SAC)

Algorithm 1 Twin Delayed DDPG

```

1: Input: initial policy parameters  $\theta$ , Q-function parameters  $\phi_1, \phi_2$ , empty replay buffer  $\mathcal{D}$ 
2: Set target parameters equal to main parameters  $\theta_{\text{targ}} \leftarrow \theta, \phi_{\text{targ},1} \leftarrow \phi_1, \phi_{\text{targ},2} \leftarrow \phi_2$ 
3: repeat
4:   Observe state  $s$  and select action  $a = \text{clip}(\mu_\theta(s) + \epsilon, a_{\text{Low}}, a_{\text{High}})$ , where  $\epsilon \sim \mathcal{N}$ 
5:   Execute  $a$  in the environment
6:   Observe next state  $s'$ , reward  $r$ , and done signal  $d$  to indicate whether  $s'$  is terminal
7:   Store  $(s, a, r, s', d)$  in replay buffer  $\mathcal{D}$ 
8:   If  $s'$  is terminal, reset environment state.
9:   if it's time to update then
10:    for  $j$  in range(however many updates) do
11:      Randomly sample a batch of transitions,  $B = \{(s, a, r, s', d)\}$  from  $\mathcal{D}$ 
12:      Compute target actions

```

$$a'(s') = \text{clip}(\mu_{\theta_{\text{targ}}}(s') + \text{clip}(\epsilon, -c, c), a_{\text{Low}}, a_{\text{High}}), \quad \epsilon \sim \mathcal{N}(0, \sigma)$$

13: Compute targets

Calculate target Q

$$y(r, s', d) = r + \gamma(1 - d) \min_{i=1,2} Q_{\phi_{\text{targ},i}}(s', a'(s'))$$

14: Update Q-functions by one step of gradient descent using

$$\nabla_{\phi_i} \frac{1}{|B|} \sum_{(s,a,r,s',d) \in B} (Q_{\phi_i}(s, a) - y(r, s', d))^2 \quad \text{for } i = 1, 2$$

Update Critic

15: **if** $j \bmod \text{policy_delay} = 0$ **then**

16: Update policy by one step of gradient ascent using

$$\nabla_{\theta} \frac{1}{|B|} \sum_{s \in B} Q_{\phi_i}(s, \mu_{\theta}(s))$$

Update Actor

17: Update target networks with

$$\begin{aligned} \phi_{\text{targ},i} &\leftarrow \rho \phi_{\text{targ},i} + (1 - \rho) \phi_i \\ \theta_{\text{targ}} &\leftarrow \rho \theta_{\text{targ}} + (1 - \rho) \theta \end{aligned} \quad \text{for } i = 1, 2$$

18: **end if**

19: **end for**

20: **end if**

21: **until** convergence

Algorithm 1 Soft Actor-Critic

```

1: Input: initial policy parameters  $\theta$ , Q-function parameters  $\phi_1, \phi_2$ , empty replay buffer  $\mathcal{D}$ 
2: Set target parameters equal to main parameters  $\phi_{\text{targ},1} \leftarrow \phi_1, \phi_{\text{targ},2} \leftarrow \phi_2$ 
3: repeat
4:   Observe state  $s$  and select action  $a \sim \pi_\theta(\cdot|s)$ 
5:   Execute  $a$  in the environment
6:   Observe next state  $s'$ , reward  $r$ , and done signal  $d$  to indicate whether  $s'$  is terminal
7:   Store  $(s, a, r, s', d)$  in replay buffer  $\mathcal{D}$ 
8:   If  $s'$  is terminal, reset environment state.
9:   if it's time to update then
10:    for  $j$  in range(however many updates) do
11:      Randomly sample a batch of transitions,  $B = \{(s, a, r, s', d)\}$  from  $\mathcal{D}$ 
12:      Compute targets for the Q functions: Calculate target Q with entropy

```

$$y(r, s', d) = r + \gamma(1 - d) \left(\min_{i=1,2} Q_{\phi_{\text{targ},i}}(s', \tilde{a}') - \alpha \log \pi_\theta(\tilde{a}'|s') \right), \quad \tilde{a}' \sim \pi_\theta(\cdot|s')$$

13: Update Q-functions by one step of gradient descent using

$$\nabla_{\phi_i} \frac{1}{|B|} \sum_{(s,a,r,s',d) \in B} (Q_{\phi_i}(s, a) - y(r, s', d))^2 \quad \text{for } i = 1, 2$$

Update Critic

14: Update policy by one step of gradient ascent using **Update Actor with entropy**

$$\nabla_{\theta} \frac{1}{|B|} \sum_{s \in B} \left(\min_{i=1,2} Q_{\phi_i}(s, \tilde{a}_\theta(s)) - \alpha \log \pi_\theta(\tilde{a}_\theta(s)|s) \right),$$

where $\tilde{a}_\theta(s)$ is a sample from $\pi_\theta(\cdot|s)$ which is differentiable wrt θ via the reparametrization trick.

15: Update target networks with

$$\phi_{\text{targ},i} \leftarrow \rho \phi_{\text{targ},i} + (1 - \rho) \phi_i \quad \text{for } i = 1, 2$$

16: **end for**

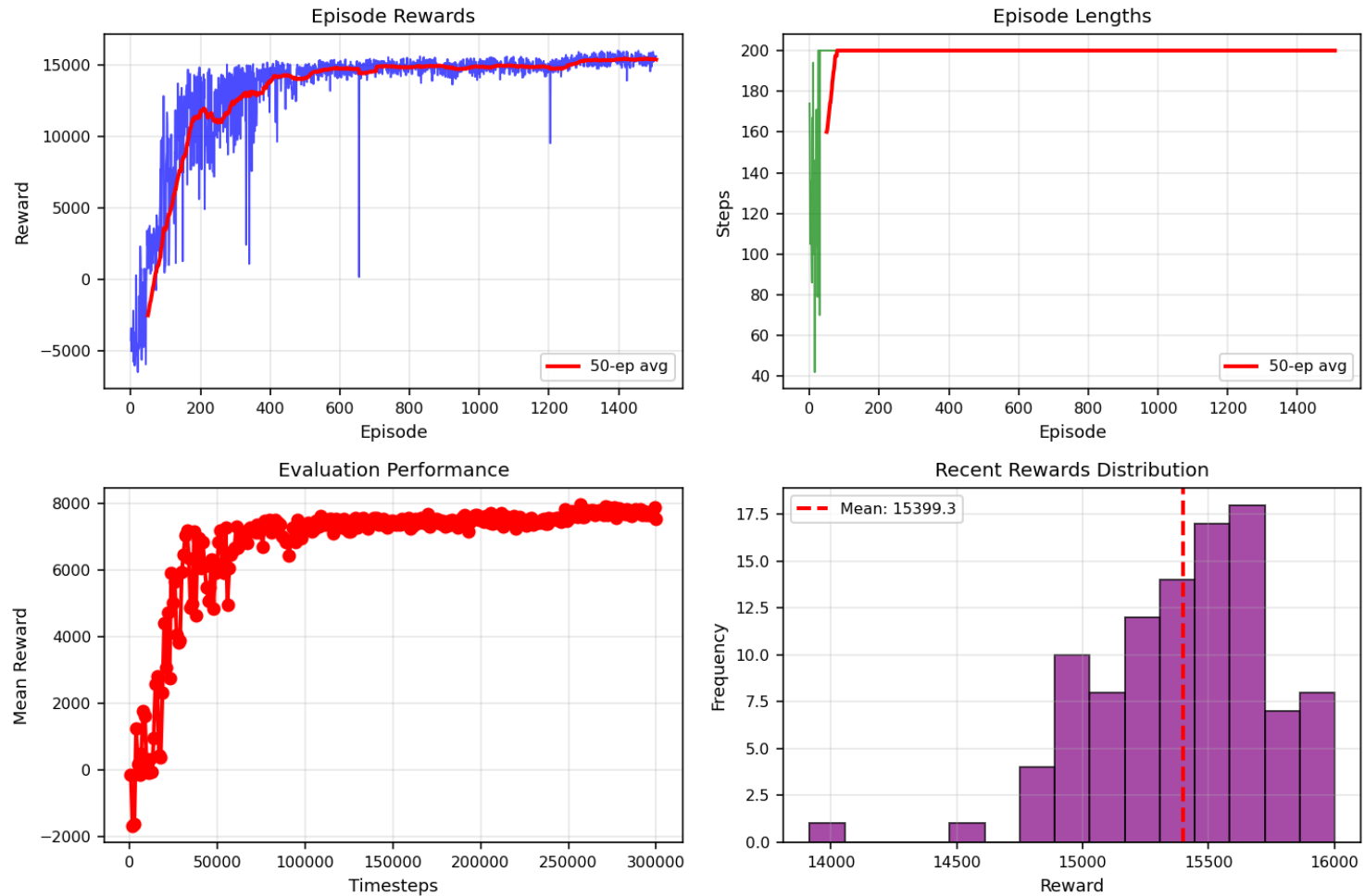
17: **end if**

18: **until** convergence



Training Result

Training Statistics | Episodes: 1509 | Timesteps: 300000



Reference

- ▶ M. Lapan, *Deep Reinforcement Learning Hands-On*, 2nd ed. Birmingham, UK: Packt Publishing, 2020.
- ▶ A. Zai and B. Brown, *Deep Reinforcement Learning in Action*. Shelter Island, NY: Manning Publications, 2020.
- ▶ S. Ravichandiran, *Deep Reinforcement Learning with Python*, 2nd ed. Birmingham, UK: Packt Publishing, 2020.